

CYBERTRACE

Digital Forensics & Incident Investigation Platform

Project type	Cybersecurity portfolio project (web application)
Stack	HTML / CSS / JavaScript, Python FastAPI, SQLAlchemy, SQLite
Architecture	Client-server with REST API
Repository	github.com/sagarsag855/Cyber-Trace

Project Report

Abstract

CyberTrace is a web-based platform for organising and managing cybersecurity investigations. It lets investigators create cases, register digital evidence, and keep investigation data in a central database. It combines a custom HTML/CSS/JavaScript frontend with a Python FastAPI backend and a SQLite database accessed through SQLAlchemy.

The current version delivers the core of case and evidence management and lays the foundation for advanced forensic features such as evidence integrity verification, event correlation, investigation timelines and automated reporting.

1. Introduction

CyberTrace was built as a cybersecurity portfolio project to demonstrate practical knowledge of web application development, backend API development, database management and digital-forensics concepts. It provides a structured environment for creating investigation cases, recording evidence and maintaining investigation information in one place.

2. Project Objectives

- Create and manage investigation cases.
- Record digital evidence associated with cases.
- Maintain structured investigation information in a database.
- Provide APIs for managing cases and evidence.
- Establish relationships between investigations and evidence.
- Provide a foundation for forensic event analysis.
- Demonstrate practical cybersecurity software-development skills.

3. Problem Statement

Digital investigations involve large amounts of information: cases, evidence, timestamps, notes and other forensic data. Managing these manually makes it hard to keep a clear link between an investigation and its evidence. CyberTrace addresses this with a centralised application where cases and evidence are organised through a structured web interface. It establishes the foundation for the following workflow:

Stage	Activity
1	Investigation case
2	Evidence collection
3	Evidence registration
4	Evidence analysis
5	Event correlation
6	Investigation timeline

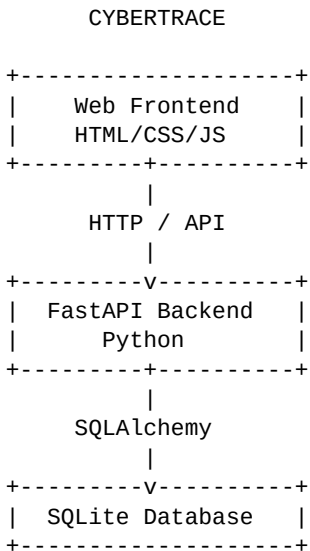
Stage	Activity
7	Final report

4. Technologies Used

Technology	Purpose
HTML	Frontend structure
CSS	User-interface styling
JavaScript	Frontend interaction
Python	Backend development
FastAPI	REST API framework
SQLAlchemy	Database ORM
SQLite	Database storage
VS Code	Development environment
Git / GitHub	Version control and project hosting

5. System Architecture

CyberTrace follows a client-server architecture. The frontend sends HTTP requests to the FastAPI backend, which uses SQLAlchemy to read and write the SQLite database.



6. Application Components

6.1 Frontend

A custom interface built with HTML, CSS and JavaScript. It lets users work with cases and evidence without touching the database directly, and is organised around the investigation workflow.

6.2 FastAPI Backend

The API layer between frontend and database. It receives requests, processes case and evidence operations, validates data, communicates with the database and returns structured responses. The design also allows additional forensic analysis services to be added later.

6.3 Database

SQLite provides persistent storage and SQLAlchemy acts as the ORM. Each case holds its case information, investigation details and associated evidence, and each evidence item carries its own information and metadata. Using an ORM also makes it easier to move to another relational database in future.

7. Case Management

Case management is a core component. Investigators create and manage individual cases, and each case acts as a central container for investigation information and its evidence, for example CASE-001 holding case information, investigation details and Evidence-001 to Evidence-003. This gives a clear relationship between an investigation and the evidence collected for it.

8. Evidence Management

Evidence must stay organised and traceable throughout an investigation. CyberTrace provides a structured way to register evidence and associate it with cases:

Step	Action
1	Case created
2	Evidence identified
3	Evidence registered
4	Evidence associated with case
5	Evidence available for analysis

This is the foundation for future integrity verification, hashing, metadata extraction and forensic analysis.

9. Investigation Workflow

The complete intended workflow runs from creating an investigation and case, through registering, organising and analysing evidence, to correlating events, building a timeline and generating a report. The current implementation covers the early stages (case creation and evidence registration and organisation); analysis, correlation, timeline and reporting are planned as future modules.

10. Digital Forensics Application

CyberTrace is designed around common forensic investigation concepts. A typical investigation involves:

- Identifying an incident and creating an investigation case.
- Collecting relevant evidence and recording its information.

- Examining evidence and identifying relevant events.
- Establishing relationships between events and creating a timeline.
- Documenting findings and generating a final report.

CyberTrace provides the application foundation for organising these activities.

11. Security Considerations

Because the platform targets cybersecurity investigations, security matters. The architecture allows the following to be added:

- User authentication and role-based access control.
- Evidence integrity verification with SHA-256 hashing.
- Audit logging, access tracking and investigation activity logs.
- Secure API endpoints.
- Evidence-chain tracking.

12. Current Features

Area	Implemented
Case management	Case creation, information storage, retrieval and organisation.
Evidence management	Evidence registration, association with cases, data storage.
Backend	FastAPI REST API, Python backend, SQLAlchemy database integration.
Database	SQLite persistence with structured case/evidence relationships.
Frontend	Custom HTML and CSS interface with JavaScript-based API interaction.

13. Project Structure

```
CyberTrace/
|-- frontend/
|   |-- HTML files
|   |-- CSS files
|   |-- JavaScript files
|-- backend/
|   |-- FastAPI application
|   |-- API routes
|   |-- database configuration
|   |-- SQLAlchemy models
|-- database/
|   |-- SQLite database
|-- README.md
```

The internal organisation can be expanded as further forensic modules are added.

14. Testing

The application was tested locally during development. Testing covered starting the FastAPI backend, loading the frontend, creating cases, adding evidence, confirming database persistence, retrieving stored information and checking frontend/backend communication. The application successfully demonstrated the basic investigation-management workflow.

15. Example Investigation Workflow

Step	Action
1	Investigator opens CyberTrace
2	Creates a new investigation case
3	Enters case information
4	Adds associated evidence
5	Evidence is stored in SQLite
6	Evidence is retrieved through the application
7	Investigation information remains organised

16. Limitations

The current version is a foundation for a larger forensic platform. Its limitations are:

- No advanced forensic file analysis or automated malware analysis.
- No complete evidence-chain management.
- No advanced event correlation or full timeline engine.
- No automated forensic report generation.
- No enterprise authentication system.
- SQLite suits development and smaller deployments better than large multi-user ones.
- It does not replace professional digital-forensics suites.

17. Future Enhancements

Enhancement	Description
Investigation timeline	A timeline engine that orders forensic events chronologically (login, file creation, process start, network connection, suspicious activity) so investigators can see the sequence.
Event analysis	Analysis and correlation of login, process, file-system, network and security events to reveal relationships between activities.
Evidence integrity	Calculate SHA-256 hashes at registration, store them, and compare on later verification to detect changes to evidence.
Report generation	Automated reports with case information, evidence inventory, timeline, key events, findings, risk assessment, investigator notes and hashes.

Enhancement	Description
Security dashboard	Interactive dashboard showing active cases, evidence, findings, the investigation timeline and recent events.

18. GitHub Deployment

CyberTrace was version-controlled with Git and published at github.com/sagarsag855/Cyber-Trace, which gives access to the source code and documentation. Publishing it also demonstrates practical use of Git repositories, branch management, commits, remote repositories and GitHub.

19. Learning Outcomes

- Digital-forensics concepts and incident investigation workflows.
- Full-stack development with HTML, CSS, JavaScript and Python.
- FastAPI and REST API development.
- SQLAlchemy, SQLite and database relationships.
- Frontend/backend communication and application debugging.
- Git, GitHub and cybersecurity software architecture.

The project also showed how cybersecurity concepts can be implemented as a practical software application rather than only through command-line tools.

20. Conclusion

CyberTrace is a cybersecurity-focused web application that supports digital forensic investigations and incident management. It combines an HTML/CSS/JavaScript frontend with a Python FastAPI backend and SQLite database, and its current implementation delivers core case and evidence management as a foundation for more advanced forensic functionality.

Future development can grow it into a fuller forensic platform through evidence integrity verification, event correlation, investigation timelines, automated reporting, advanced analysis and an interactive dashboard. Overall, CyberTrace is a practical portfolio project showing the ability to design and build a structured digital-forensics application from the ground up.